ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ «ВЫСШАЯ ШКОЛА ЭКОНОМИКИ» ФАКУЛЬТЕТ КОМПЬЮТЕРНЫХ НАУК

Ахундов Алексей Назимович, Жарковский Дмитрий Андреевич

МАГИСТЕРСКАЯ ДИССЕРТАЦИЯ

Генерация персонализированных учебных конспектов по предметам ЕГЭ на основе языковых моделей и пользовательской аналитики

Generation of Personalized Educational Notes for Unified State Exam Subjects Based on Language Models and User Analytics

по направлению подготовки 01.04.02 Прикладная математика и информатика
образовательная программа «Искусственный интеллект»

Студент

_____

А.Н. Ахундов

Студент

_____

Д.А. Жарковский

Научный руководитель
Кандидат физико-математических наук

_____

Е.О. Кантонистова

Соруководитель
Преподаватель

_____

П.И. Ахтямов

Москва 2026

Содержание

Аннотация **4**

Abstract **5**

Введение **6**

1 Глава 1. Проблематика персонализированного онлайн-обучения и постановка задачи генерации учебных конспектов **7**

1.1 Цифровизация подготовки к ЕГЭ и особенности рынка онлайн-обучения . . . 7

1.2 Научно-прикладные подходы к персонализации обучения 8

1.3 Критический анализ существующих решений и исследований 10

1.4 Постановка задачи ВКР и методы ее решения 11

1.5 Выводы по главе 14

2 Глава 2. Проектирование и реализация сервиса генерации персонализированных учебных конспектов **15**

2.1 Архитектура сервиса персонализированных конспектов 15

2.2 Обоснование выбора методов и инструментальных средств 18

2.3 Алгоритм формирования персонализированного конспекта 19

2.4 Реализация сервиса и решения, обеспечивающие масштабируемость 21

2.5 Результат решения задачи и личный вклад участников проекта 22

2.6 Выводы по главе 23

3 Глава 3. Реализация подсистемы извлечения и генерации персонализированных конспектов **24**

3.1 Структура предметной базы знаний 24

3.2 Векторное представление и индексирование учебных материалов 26

3.3 Гибридный поиск и многоуровневое ранжирование 27

3.4 Генерация и верификация конспекта 32

3.5 Выводы по главе 33

4 Примеры **35**

4.1 Ссылки на статьи 35

4.2 Рисунки 35

4.3 Таблицы 36

4.4 Формулы 36

Список литературы **37**

А Пример сгенерированного конспекта **41**

Аннотация

Подготовка к единому государственному экзамену (ЕГЭ) является одной из наиболее востребованных областей онлайн-образования. Существующие образовательные платформы предлагают тренажёры, видеоразборы и рекомендательные системы, однако персонализация в них, как правило, ограничивается выбором курса. При этом теоретический материал остаётся универсальным: ученик, допускающий устойчивые ошибки, вынужден самостоятельно выстраивать объяснение из разрозненных источников. Проблема усугубляется тем, что большие языковые модели (Large Language Models, LLM), способные генерировать связные объяснения, без внешних ограничений подвержены галлюцинациям и не привязаны к конкретной учебной программе. На данный момент в литературе отсутствуют решения, объединяющие пользовательскую аналитику, управляемую генерацию текста и предметную базу знаний для автоматического создания персонализированных учебных конспектов для заданий ЕГЭ. В данной работе разработан сервис для действующей веб-платформы «ЕГЭ Прорыв» [33], который на основе истории ошибок, динамики прогресса и уровня освоения навыков конкретного ученика автоматически формирует индивидуальный учебный конспект. Используя наш подход, нам удалось получить связные персонализированные конспекты, содержащие объяснения ключевых понятий, разбор типичных ошибок и рекомендации, адаптированные к дефицитам знаний обучающегося. Таким образом, полученные результаты могут быть использованы для повышения эффективности подготовки к ЕГЭ на образовательных платформах, а предложенный подход применим к другим предметам и экзаменационным форматам.

Ключевые слова

Персонализированное обучение, генерация учебных конспектов, ЕГЭ, большие языковые модели в образовании, аналитика обучения

Abstract

Preparation for the Unified State Examination is among the most sought-after areas of online education. Existing educational platforms offer drills, video walkthroughs, and recommender systems; however, personalization is usually limited to course selection. Theoretical material remains generic: a student who makes persistent errors must assemble explanations from scattered sources on their own. The problem is compounded by the fact that large language models (LLM), while able to produce coherent explanations, are prone to hallucinations without external constraints and are not tied to a specific curriculum. At present, the literature offers no solutions that combine user analytics, controlled text generation, and a subject knowledge base for the automatic creation of personalized study notes for Unified State Examination tasks. This work presents a service for the live web platform Exambreakout [33], which automatically builds an individual study note from a student's error history, progress dynamics, and skill mastery. Using our approach, we obtained coherent personalized notes containing explanations of key concepts, analyses of typical errors, and recommendations adapted to the learner's knowledge gaps. Thus, the results can improve the effectiveness of Unified State Examination preparation on educational platforms, and the proposed approach extends to other subjects and examination formats.

Keywords

Personalized learning, study note generation, Unified State Examination, large language models in education, learning analytics

Введение

Текст введения.

1 Глава 1. Проблематика персонализированного онлайн-обучения и постановка задачи генерации учебных конспектов

1.1 Цифровизация подготовки к ЕГЭ и особенности рынка онлайн-обучения

Подготовка к ЕГЭ за последние годы стала одной из наиболее заметных областей цифрового образования. На рынке одновременно присутствуют крупные образовательные платформы и специализированные онлайн-школы, предлагающие курсы, тренажеры, видеоразборы и консультации по предметам экзамена. Это подтверждается как распространенностью профильных сервисов, так и разнообразием их продуктовых моделей: отдельные предметные курсы и тренажеры представлены, например, у Яндекса, ЕГЭLand, Умскул и MAXIMUM [18, 32, 35, 36]. Для пользователя такой рынок создает широкий выбор форматов, однако одновременно усиливает конкуренцию по качеству контента, степени персонализации и скорости адаптации материалов под конкретные образовательные цели.

Современные исследования онлайн-образования показывают, что эффективность цифрового обучения определяется не только наличием учебных материалов, но и качеством структуры курса, уровнем интерактивности, развитием саморегуляции обучающегося, цифровой грамотностью и институциональной поддержкой [11]. Иными словами, простой перенос теории и задач в веб-интерфейс еще не делает обучение по-настоящему эффективным. Особенно остро это проявляется в экзаменационной подготовке, где учащемуся нужен не просто доступ к контенту, а своевременное объяснение именно тех тем, которые вызывают устойчивые ошибки.

Для подготовки к ЕГЭ эта проблема имеет особую специфику. Во-первых, содержание обучения жестко ограничено кодификатором и спецификацией экзамена, поэтому образовательный сервис должен обеспечивать релевантность и нормативную корректность материалов. Во-вторых, темп усвоения тем у разных учеников заметно различается: один обучающийся испытывает трудности с базовыми определениями, другой — с типовыми преобразованиями, третий — с переносом знаний на задания повышенной сложности. В-третьих, массовые онлайн-курсы обычно строятся вокруг усредненной траектории: ученик получает одинаковую теорию, общий набор практики и ограниченную адаптацию, выраженную чаще всего в рекомендации следующего урока или задания.

Таким образом, на рынке онлайн-подготовки сформировалось противоречие. С одной стороны, цифровые платформы обеспечивают масштабируемость, доступность и богатую учебную аналитику. С другой стороны, они по-прежнему редко предоставляют персонализированный теоретический материал в форме связного учебного конспекта, отражающего историю ошибок, текущий уровень подготовки и учебные цели конкретного ученика. Именно это противоречие и определяет актуальность разработки специализированного сервиса для платформы ЕГЭПрорыв [33]. Платформа уже содержит действующую систему адаптивного обучения на основе графа микромодулей, байесовской оценки компетенций и кривой забывания [34], а также подсистему оптимизации образовательных траекторий с применением машинного обучения [31]. Однако указанные компоненты управляют *выбором* следующего задания, но не *содержанием* теоретического объяснения, что создаёт нишу для отдельного RAG-сервиса генерации персонализированных конспектов.

1.2 Научно-прикладные подходы к персонализации обучения

В научной литературе персонализированное обучение понимается как организация учебного процесса, при которой содержание, темп, форма представления материала и обратная связь адаптируются под особенности конкретного обучающегося [15]. Такой подход противопоставляется модели “one-size-fits-all”, в которой один и тот же учебный материал предъявляется всей аудитории без учета различий в исходной подготовке, стиле усвоения и динамике прогресса.

Методологической основой персонализации выступают learning analytics и educational data mining. По определению R. Baker, образовательный data mining ориентирован на извлечение закономерностей из учебных данных для понимания и оптимизации обучения [4]. В более позднем обзоре C. Romero и S. Ventura подчеркивается, что learning analytics и educational data mining сегодня образуют единую исследовательскую область, в которой анализ цифровых следов обучающегося используется для прогнозирования результатов, выявления трудностей и поддержки педагогических решений [26]. Систематический обзор E. T. Khor и Mutthulakshmi K. показывает, что learning analytics способствует персонализации как минимум по пяти направлениям: сбор данных о прогрессе, классификация учащихся, построение циклов обратной связи, прогнозирование результатов и предоставление преподавателю или системе интерпретируемых рекомендаций в реальном времени [15].

Отдельное место в персонализации занимает моделирование знаний обучающегося. Классической работой в этой области считается статья A. Corbett и J. Anderson, где была

предложена модель knowledge tracing для оценки вероятности усвоения навыка по последовательности ответов ученика [7]. Дальнейшее развитие направление получило в работе C. Piech и соавторов о deep knowledge tracing, где динамика овладения знаниями моделируется с помощью нейросетевых методов [21]. Современный обзор G. Abdelrahman и соавторов показывает, что learner modeling и knowledge tracing остаются центральными инструментами адаптивных обучающих систем, особенно в задачах прогнозирования ошибок, выбора индивидуальной траектории и адресной поддержки [1].

Новый этап развития персонализации связан с большими языковыми моделями. Работы по BERT, GPT и LLaMA показали, что крупные трансформерные модели способны решать широкий круг задач понимания и генерации текста, включая объяснение понятий, переформулирование материала и адаптацию стиля изложения [9, 5, 28]. Для образования это особенно важно, поскольку учебный контент является прежде всего текстовым знанием, которое должно быть представлено в понятной, логичной и методически корректной форме.

При этом использование больших языковых моделей (Large Language Models, LLM) в образовании нельзя сводить к обычному чат-интерфейсу. Обзор E. Kasneci и соавторов показывает, что большие языковые модели открывают значительные возможности для тьюторской поддержки, генерации объяснений и автоматизации обратной связи, однако одновременно создают риски недостоверности, избыточной уверенности модели и педагогически неудачных подсказок [13]. Систематический обзор L. Yan и соавторов дополняет этот вывод практическими и этическими ограничениями: зависимость обучающихся от подсказок ИИ, непрозрачность алгоритмических решений, проблемы приватности и справедливости, а также сложность надежной оценки качества сгенерированного материала [30]. Более свежий систематический обзор Y. Shi и соавторов подтверждает, что LLM уже активно применяются в интеллектуальных обучающих системах, но их положительный эффект оказывается неоднородным и сильно зависит от сценария внедрения, глубины педагогической интеграции и наличия внешнего контроля [27].

Для повышения надежности генерации используются retrieval-augmented generation и агентные архитектуры. Подход Генерации с дополненной выборкой (Retrieval Augmented Generation, RAG) предполагает дополнение генерации внешним контекстом из базы знаний, что снижает вероятность галлюцинаций и делает ответы более привязанными к предметной области [17, 12]. В образовательном контексте, как показывает обзор Y. Ding и соавторов, RAG особенно полезен тогда, когда необходимо связать генерацию с учебной программой, проверенными материалами и профилем ученика [10]. Агентные системы, в свою очередь, позволяют разнести функции анализа, планирования, генерации и проверки между несколь-

кими взаимосвязанными компонентами, а библиотека LangGraph предназначена для построения управляемых stateful-пайплайнов такого типа [16]. Для образовательных решений это означает возможность перейти от одиночного ответа модели к воспроизводимому конвейеру подготовки учебного материала.

1.3 Критический анализ существующих решений и исследований

Существующие решения в предметной области можно условно разделить на три класса: массовые онлайн-курсы и платформы подготовки к экзаменам, интеллектуальные тьюторские системы на основе аналитики обучения и современные LLM-ориентированные AI-ассистенты.

Первый класс представлен образовательными платформами, ориентированными на массовый сегмент подготовки к ЕГЭ [36, 32, 35, 18]. Их сильными сторонами являются масштабируемость, насыщенная медиатека, наличие тренажеров и готовых учебных траекторий. Однако такие решения обычно адаптируют обучение на уровне выбора курса, темы или набора задач, а не на уровне глубокой реконфигурации самого объяснения учебного материала. В результате теория остается относительно универсальной, а ученик с повторяющимися ошибками нередко вынужден самостоятельно собирать для себя “личный конспект” из видео, заметок, чатов и разрозненных комментариев преподавателя.

Второй класс включает системы, основанные на learner modeling, knowledge tracing и учебной аналитике [7, 21, 1, 15]. Эти подходы хорошо решают задачи диагностики: они позволяют определить вероятность усвоения навыка, выявить “узкие места” и спрогнозировать риск неуспешности. Их принципиальное ограничение состоит в том, что они отвечают на вопрос “что именно ученик знает плохо”, но не решают в полной мере вопрос “как именно представить этому ученику новый материал”. Иными словами, аналитика эффективно обнаруживает дефицит, но сама по себе не создает качественное персонализированное объяснение.

Третий класс — это LLM-ориентированные образовательные ассистенты и AI-тьюторы. На прикладном уровне к наиболее заметным примерам относятся Khanmigo и Q-Chat [14, 24]. На научном уровне активно развиваются работы по генерации персонализированной обратной связи и рекомендаций с помощью LLM [20, 29]. Достоинство этих решений заключается в естественности взаимодействия и способности адаптировать объяснение к запросу пользователя. Тем не менее большинство таких систем по-прежнему работает в реактивном диалоговом режиме: они хорошо отвечают на вопрос, но хуже формируют целостный учебный

текст, который последовательно раскрывает тему, объясняет проблемные места, удерживает необходимую глубину и при этом не выходит за рамки предметного стандарта.

Критический анализ литературы позволяет выделить несколько нерешенных проблем.

1 **Недостаточная персонализация теоретического материала.** Обзоры по learning analytics показывают, что данные о прогрессе ученика могут использоваться для классификации, прогнозирования и формирования обратной связи [15]. Однако в массовых онлайн-курсах эти данные редко преобразуются в автоматически создаваемый учебный конспект, который учитывает именно слабые темы конкретного обучающегося.

2 **Фрагментарность чатового формата.** LLM-тьюторы удобны для диалога, но учебный процесс при подготовке к ЕГЭ требует не только ответов на локальные вопросы, но и системного объяснения темы в логически связанной форме. По этой причине чат-ориентированный интерфейс не всегда закрывает потребность в структурированном повторении материала [13, 27].

3 **Риски недостоверности и педагогической несогласованности.** Без внешних ограничений LLM могут генерировать правдоподобный, но методически неудачный или частично некорректный текст. Для образования это критично, поскольку ошибка в объяснении быстро тиражируется и закрепляется у обучающегося [30, 10].

4 **Недостаточная прозрачность и доверие к персонализирующим алгоритмам.** Human-centered обзоры по LA/AI in education показывают, что успешное внедрение образовательного ИИ требует учета вопросов контроля человека, надежности, безопасности и доверия пользователей, а также внятного обоснования того, почему система предлагает именно такой материал [2].

Следовательно, перспективным направлением является объединение трех линий развития: учебной аналитики как источника персонализирующего сигнала, LLM как механизма генерации содержательного текста и агентной оркестрации как способа сделать генерацию многошаговой, контролируемой и воспроизводимой. Именно такая интеграция позволяет перейти от универсального цифрового курса и реактивного чат-бота к сервису, создающему персонализированный учебный конспект как самостоятельный образовательный артефакт.

1.4 Постановка задачи ВКР и методы ее решения

Объектом исследования в данной работе является процесс персонализированной подготовки обучающихся к ЕГЭ в цифровой образовательной среде. Предмет исследования —

методы и программные средства генерации учебных конспектов, адаптированных к индивидуальным особенностям обучающегося на основе данных пользовательской аналитики и возможностей больших языковых моделей.

Целью выпускной квалификационной работы является разработка сервиса для веб-ресурса ЕГЭПрорыв [33], который автоматически формирует персонализированные учебные конспекты по предметам ЕГЭ с учетом текущего уровня подготовки ученика, его истории ошибок и динамики освоения тем.

Исходя из проведенного обзора, для достижения поставленной цели необходимо решить следующие задачи:

1 проанализировать существующие подходы к персонализации обучения, генерации учебных материалов и использованию LLM в образовательных сервисах;

2 определить состав пользовательской аналитики, достаточной для адаптации содержания конспекта: изученные темы, типовые ошибки, показатели успеваемости, частоту повторных затруднений и уровень освоения навыков;

3 разработать структуру конспекта, в которой глубина объяснения, выбор примеров, акценты на типичных ошибках и объем повторения зависят от профиля конкретного ученика;

4 спроектировать агентную систему генерации с использованием LangChain и планируемой интеграцией LangGraph, разделяющую этапы анализа входных данных, планирования структуры конспекта, генерации текста и внутренней проверки результата [16];

5 обеспечить управляемость генерации за счет внешних ограничений предметной области, в том числе опоры на учебные материалы, специфику экзамена и контролируемую подачу контекста модели [17, 10];

6 определить критерии качества получаемого конспекта: релевантность теме, уровень персонализации, логичность структуры, читаемость, фактологическая корректность и пригодность для практического использования в учебном процессе.

Разрабатываемый сервис должен принимать на вход тему или раздел предмета ЕГЭ, профиль обучающегося и агрегированные данные о его взаимодействии с платформой. В качестве входной аналитики могут выступать результаты решения заданий, сведения о частоте

ошибок по конкретным типам задач, история прохождения тем, показатели повторного обращения к материалу и иные характеристики, отражающие реальное состояние освоения курса. Выходом системы должен быть связный конспект, который содержит краткое введение в тему, адаптированное объяснение ключевых понятий, разбор типичных ошибок, примеры решения и рекомендации по дальнейшему изучению.

Метод решения поставленной задачи основан на сочетании трех компонентов. Первый компонент — learner modeling и learning analytics, необходимые для построения информативного профиля ученика и выявления содержательных пробелов [15, 1]. Второй компонент — большая языковая модель, обеспечивающая генерацию связного педагогически ориентированного текста [13, 27]. Третий компонент — агентная orchestration-логика, проектируемая с учётом интеграции LangGraph поверх LangChain, обеспечивающая разделение ролей внутри конвейера генерации, сохранение состояния и управляемое прохождение этапов анализа, планирования и синтеза ответа [16].

С практической точки зрения разрабатываемая система должна удовлетворять следующим ограничениям:

1 **предметная корректность** — содержание конспекта не должно выходить за рамки школьной программы и формата ЕГЭ;

2 **персонализированность** — объяснение должно учитывать реальные пробелы ученика, а не ограничиваться формальным упоминанием его уровня;

3 **прозрачность логики** — выбор акцентов в конспекте должен быть объясним через доступные аналитические признаки;

4 **безопасность** — система не должна раскрывать чувствительные пользовательские данные и должна минимизировать риск генерации недостоверных рекомендаций;

5 **интегрируемость** — сервис должен быть встроен в уже существующий веб-ресурс без разрушения его текущих рабочих сценариев.

Итак, постановка задачи ВКР может быть сформулирована следующим образом: требуется разработать программный сервис, который на основе пользовательской аналитики и агентно организованной языковой модели генерирует персонализированные учебные конспекты по предметам ЕГЭ для платформы ЕГЭПрорыв [33], обеспечивая адаптацию содержания к индивидуальным дефицитам знаний, соблюдение предметных ограничений и пригодность результата для реального учебного использования.

1.5 Выводы по главе

1 Рынок онлайн-подготовки к ЕГЭ предлагает масштабируемые цифровые курсы и тренажеры, однако в большинстве решений персонализация затрагивает преимущественно выбор заданий и траектории, а не автоматическое создание индивидуализированного теоретического материала.

2 Learning analytics, educational data mining и knowledge tracing дают теоретическую и инструментальную основу для выявления учебных дефицитов, но сами по себе не решают задачу генерации связного учебного конспекта.

3 Большие языковые модели открывают возможность автоматизированного создания адаптивных образовательных текстов, однако требуют внешних ограничений, аналитического контекста и контролируемой оркестрации из-за рисков недостоверности, непрозрачности и педагогической несогласованности.

4 Перспективным решением является сервис, интегрированный в ЕГЭПрорыв [33], который объединяет пользовательскую аналитику, LLM и агентную архитектуру на базе LangChain с планируемым переходом на LangGraph для генерации персонализированных конспектов по предметам ЕГЭ.

2 Глава 2. Проектирование и реализация сервиса генерации персонализированных учебных конспектов

2.1 Архитектура сервиса персонализированных конспектов

Разрабатываемый сервис встроен в цифровую образовательную платформу ЕГЭПрорыв [33], на которой пользователь уже взаимодействует с учебными материалами, решает задания, получает обратную связь и формирует цифровой след своей подготовки. Ядро платформы составляет система адаптивного обучения, использующая граф микромодулей, байесовскую оценку компетенций и кривую забывания для построения индивидуальной образовательной траектории [34, 31]. Поскольку в платформе присутствуют сценарии автоматизированной практики, проверки решений и генерации задач, в том числе задач с графиками, новый модуль персонализированных конспектов проектировался не как изолированный прототип, а как отдельный прикладной сервис, использующий общую пользовательскую и предметную инфраструктуру платформы.

Архитектурно решение построено по принципу разделения интерфейсного, прикладного, аналитического и интеллектуального контуров. Такой подход позволяет не смешивать работу пользовательского интерфейса, бизнес-логику платформы, хранение учебных данных и ресурсоемкую генерацию текста. В результате система сохраняет расширяемость и может быть встроена в существующий веб-ресурс без разрушения уже работающих пользовательских сценариев.

В состав сервиса входят следующие основные компоненты (Рисунок 2.1).

1 **Веб-интерфейс платформы.** Обеспечивает инициацию запроса на генерацию конспекта, передачу параметров темы и отображение результата пользователю.

2 **Backend-сервис оркестрации.** Принимает запрос, получает учебную аналитику, формирует поисковые и генеративные запросы, вызывает внешние интеллектуальные компоненты и возвращает итоговый конспект.

3 **Агентный контур (проектируется на базе LangGraph).** Управляет последовательностью состояний внутри пайплайна генерации: анализирует входные данные, координирует retrieval, планирует структуру конспекта, передает данные на этап генерации и инициирует проверку результата [16].

4 **Контур пользовательской аналитики.** Содержит сведения о прогрессе ученика:

историю решенных заданий, типовые ошибки, темы с повторяющимися затруднениями, показатели успешности и иные признаки, необходимые для персонализации.

5 **Подсистема базы знаний.** Хранит учебные материалы, фрагменты теории, примеры и структурированные предметные сведения, используемые как доверенный внешний контекст для генерации.

6 **Векторная база данных Qdrant.** Используется для семантического поиска по векторным представлениям учебных фрагментов и поддерживает быстрый retrieval релевантного контекста для RAG-пайплайна [23].

7 **LLM-модуль генерации.** Формирует итоговый текст конспекта на основе темы, пользовательского профиля, найденного контекста и заранее заданной структуры ответа.

8 **Инфраструктурные компоненты сопровождения.** Для промышленной эксплуатации архитектура предусматривает кэширование часто используемых данных, фоновые задачи, журналирование и мониторинг, что особенно важно для высоконагруженного образовательного сервиса.

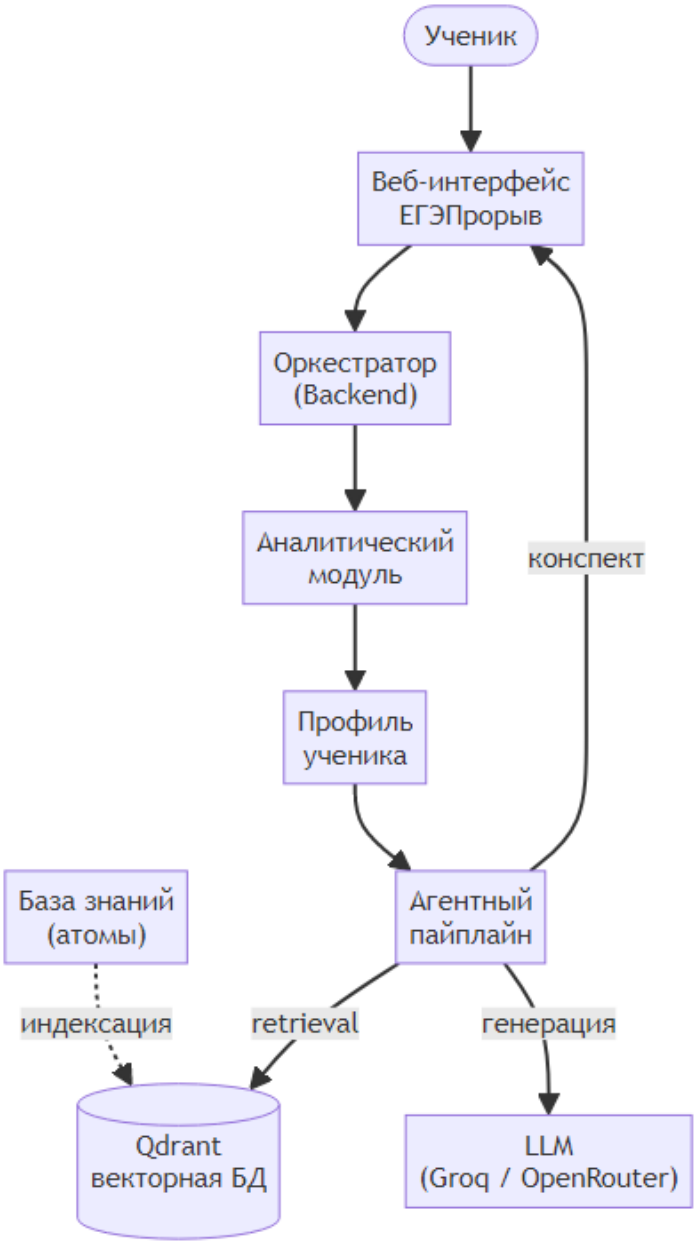


Рис. 2.1: Компонентная архитектура сервиса персонализированных конспектов.

Поток обработки данных в системе выглядит следующим образом. Сначала пользователь инициирует запрос на конспект по конкретной теме. Затем backend-сервис получает агрегированные сведения о текущем состоянии подготовки ученика и преобразует их в компактный профиль персонализации. Далее управление передается в агентный пайплайн (с перспективой миграции на LangGraph), где отдельные шаги выполняются как согласованные состояния: извлечение аналитики, retrieval релевантного контекста, планирование структуры будущего конспекта, генерация текста и проверка полноты результата. Только после этого готовый ответ проходит постобработку и возвращается в интерфейс платформы.

Таким образом, архитектура сервиса сочетает три принципиально разные группы дан-

ных: предметный контекст, пользовательскую аналитику и генеративную модель. Именно их совместное использование позволяет перейти от универсального справочного текста к индивидуализированному конспекту, ориентированному на дефициты знаний конкретного обучающегося.

2.2 Обоснование выбора методов и инструментальных средств

Выбор методов решения определялся спецификой задачи. С одной стороны, системе требовалось генерировать естественный, логически связный и педагогически понятный текст. С другой стороны, содержание такого текста должно было оставаться привязанным к школьной программе, формату ЕГЭ и истории ошибок конкретного пользователя. По этой причине в работе был выбран комбинированный подход, объединяющий учебную аналитику, retrieval-augmented generation, векторный поиск и агентную оркестрацию с планируемым переходом на LangGraph.

Использование *learning analytics* и элементов learner modeling позволяет извлекать из цифрового следа ученика признаки, значимые для персонализации: устойчивые ошибки, неусвоенные темы, характер затруднений и динамику прогресса [15, 1]. Без этого этапа генерация оставалась бы лишь тематической, но не персонализированной. Следовательно, аналитический контур является не вспомогательной, а содержательно определяющей частью решения.

Выбор RAG объясняется необходимостью снизить вероятность галлюцинаций и сделать итоговый конспект опирающимся на проверенные учебные материалы [17, 12, 10]. В отличие от прямого обращения к LLM без внешнего контекста, RAG-подход позволяет перед генерацией извлекать фрагменты теории, примеры и формулировки, релевантные теме и уровню обучающегося. Для образовательного сервиса это критично, поскольку даже стилистически хороший, но предметно неточный текст не обладает практической ценностью.

В качестве векторной базы данных был выбран Qdrant. Такое решение обусловлено тем, что сервису требуется выполнять не обычный поиск по ключевым словам, а семантический retrieval по фрагментам учебных материалов. Qdrant предоставляет модель хранения коллекций векторов с метаданными, поддерживает payload-фильтрацию, масштабирование retrieval-слоя и подходит для задач RAG, где важны как релевантность, так и задержка ответа [23]. По сравнению с хранением материалов только в реляционной форме это дает более устойчивый поиск близких по смыслу фрагментов, даже если формулировки ученического запроса и исходного учебного текста различаются лексически.

Языковая модель в данной архитектуре используется не как автономный источник знания, а как механизм генеративной сборки итогового текста. Ее задача заключается в том, чтобы на основе уже найденного контекста и профиля ученика сформировать конспект с нужной глубиной объяснения, последовательностью разделов и акцентом на трудных местах. Такой подход соответствует выводам современных исследований об эффективном использовании LLM в образовании: наибольшую практическую пользу дают не “свободные” ответы модели, а управляемые сценарии с внешними ограничениями и прозрачной логикой формирования контекста [13, 30, 27].

Выбор LangGraph в качестве целевого фреймворка оркестрации обусловлен необходимостью формализовать многошаговую агентную логику генерации: отдельные стадии — анализ профиля ученика, retrieval контекста, построение плана конспекта, генерация текста и контроль структуры — должны быть представлены не как набор слабо связанных вызовов, а как единый stateful-граф с явным переходом между состояниями [16]. В текущей реализации пайплайн реализован как последовательность связанных вызовов Python-функций; планируется перенос оркестрации на LangGraph для повышения воспроизводимости и управляемости. Это упрощает сопровождение сервиса, повышает воспроизводимость результата и позволяет точнее контролировать, на каком именно этапе формируется персонализация.

С инженерной точки зрения выбранный набор средств также оправдан. Backend-оркестрация позволяет отделить пользовательский интерфейс от генеративной логики, а модульная организация сервиса облегчает дальнейшее масштабирование, замену отдельных компонентов и контроль качества результата. Иными словами, предложенный стек является не только функционально достаточным, но и технологически пригодным для интеграции в действующую образовательную платформу.

2.3 Алгоритм формирования персонализированного конспекта

Алгоритм работы сервиса можно представить как последовательность взаимосвязанных этапов, на каждом из которых происходит уточнение будущего результата.

1 **Прием пользовательского запроса.** Система получает идентификатор пользователя, тему или раздел ЕГЭ и параметры сценария генерации.

2 **Сбор аналитического профиля.** Из пользовательского контура извлекаются результаты решения заданий, частота ошибок, история повторений, показатели успеваемости и иные признаки, характеризующие реальный уровень освоения темы.

3 **Формирование персонализирующего представления.** Полученные признаки преобразуются в компактный профиль: какие понятия требуют развернутого объяснения, какие типы ошибок нужно разобрать, а какие элементы можно дать кратко.

4 **Передача в агентный пайплайн.** Оркестратор переводит обработку в последовательность специализированных шагов, задающих порядок прохождения следующих этапов и сохраняющих состояние пайплайна между ними (с перспективой представления их как узлов LangGraph).

5 **Извлечение предметного контекста.** По теме и аналитическому профилю формируется запрос к базе знаний; затем в Qdrant находятся наиболее релевантные фрагменты теории, определения, формулы, примеры и пояснения.

6 **Планирование структуры конспекта.** На основе профиля ученика и retrieved- контекста формируется план будущего материала: порядок разделов, глубина объяснения, обязательные акценты на типичных ошибках и набор примеров.

7 **Сборка промпта.** Backend-сервис объединяет тему, ограничения предметной области, профиль ученика и найденные фрагменты контекста в единый структурированный запрос к модели.

8 **Генерация конспекта.** Языковая модель формирует связный текст, в котором выдерживаются введение в тему, объяснение ключевых понятий, разбор типичных ошибок, примеры и рекомендации по дальнейшему изучению.

9 **Внутренняя проверка результата.** Агентный контур контролирует наличие обязательных смысловых блоков и при необходимости повторно передает данные на этап генерации или доуточнения плана.

10 **Постобработка результата.** Выполняется проверка структуры ответа, целостности разделов и соответствия требуемому формату вывода.

11 **Возврат результата пользователю.** Итоговый конспект отображается в интерфейсе платформы и может быть использован как самостоятельный учебный материал.

Ключевым отличием предложенного алгоритма от простого вызова LLM является наличие промежуточных стадий анализа, retrieval и агентной координации (с перспективой перехода на LangGraph). Модель не генерирует текст “с нуля”, а работает в условиях контролируемого контекста и заранее заданной логики персонализации. Это позволяет сделать

конспект не только связным, но и объяснимым с точки зрения причин выбора тех или иных акцентов.

Содержательно алгоритм решает сразу две задачи. Первая задача состоит в выборе того, *что именно* следует объяснять конкретному ученику. Вторая задача заключается в определении того, *как именно* это объяснение должно быть представлено: подробно или кратко, через определение, пример, предупреждение о типичной ошибке или через повторение базового факта. Именно сочетание аналитического и генеративного контуров позволяет получить результат, соответствующий образовательной логике сервиса.

2.4 Реализация сервиса и решения, обеспечивающие масштабируемость

Так как ВКР относится к программным проектам, ориентированным на создание работоспособного и потенциально высоконагруженного приложения, при проектировании сервиса необходимо учитывать не только качество генерации, но и эксплуатационные характеристики решения. Для образовательной платформы это особенно важно, поскольку обращения к сервису могут происходить массово: перед экзаменом возрастает интенсивность повторения тем, пользователи параллельно решают задания, просматривают аналитику и запрашивают новые материалы.

Базовая производительность сервиса достигается уже за счет самой логики RAG. Учебные материалы индексируются заранее, а в момент пользовательского запроса система выполняет retrieval только по готовому векторному представлению. Это существенно дешевле и быстрее, чем каждый раз передавать в модель большой объем “сырого” текста. Следовательно, вынесение базы знаний в Qdrant одновременно повышает релевантность контекста и снижает вычислительную нагрузку на генеративный слой.

Для снижения времени отклика и уменьшения числа повторных обращений к тяжелым компонентам архитектура сервиса должна включать кэширование часто используемых сущностей: шаблонов промптов, фрагментов учебного контекста, промежуточных аналитических представлений и результатов повторных запросов по близким темам. Такое решение особенно эффективно в образовательном домене, где большое число пользователей обращается к сходным разделам программы и типовым ошибкам.

Следующим важным решением является разделение интерактивного и фонового контуров обработки. Запрос пользователя должен быстро пройти валидацию, получить идентификатор задачи и быть передан в обработку без блокировки интерфейса. Ресурсоемкие

операции — генерация длинного текста, формирование эмбеддингов для новых материалов, обновление индексов, агрегация журналов — рационально выполнять асинхронно. Это позволяет не перегружать основной web-контур и поддерживать стабильную работу платформы даже при росте числа одновременных пользователей.

Для масштабирования по числу запросов прикладной слой сервиса целесообразно сохранять stateless. В этом случае горизонтальное масштабирование backend-экземпляров, балансировка входящего трафика и вынос состояния в специализированные хранилища становятся естественным продолжением архитектуры. Аналогично retrieval-слой на базе Qdrant может масштабироваться отдельно от web-слоя и LLM-модуля, что важно при неравномерной нагрузке, когда число поисковых обращений существенно превышает число генеративных вызовов.

Не менее значимым является контур наблюдаемости. Для промышленной эксплуатации необходимо собирать метрики времени ответа, частоты ошибок, загрузки очередей, числа обращений к модели, качества retrieval и доли неуспешных генераций. Эти данные позволяют не только поддерживать стабильность сервиса, но и совершенствовать персонализацию на основе реальных пользовательских сценариев. Таким образом, масштабируемость в данной работе понимается не как единичное аппаратное усиление, а как архитектурное свойство системы, обеспечиваемое раздельным масштабированием компонентов, кэшированием, асинхронной обработкой и постоянным мониторингом.

2.5 Результат решения задачи и личный вклад участников проекта

Результатом выполненной работы является архитектурно и методически оформленный сервис генерации персонализированных учебных конспектов, интегрируемый в платформу ЕГЭПрорыв [33]. Данный сервис использует учебную аналитику пользователя, retrieval по базе знаний и большую языковую модель для автоматического формирования связного учебного текста, адаптированного к текущему уровню подготовки обучающегося.

Здесь будет описание того, что сделал каждый участник проекта...

Практическая ценность разработанного решения состоит в том, что оно позволяет перевести персонализацию теоретического материала из ручного режима в автоматизированный. Платформа получает возможность не только фиксировать ошибки пользователя и подбирать ему задания, но и формировать индивидуальный теоретический материал, непосредственно направленный на устранение обнаруженных дефицитов. Тем самым повышается полезность накопленной учебной аналитики и расширяется набор образовательных сценари-

ев, доступных пользователю в одном цифровом контуре.

Научно-прикладная новизна решения заключается в интеграции нескольких подходов в едином сервисе: аналитики учебных данных, retrieval-augmented generation и векторного поиска по предметной базе знаний. В отличие от типичного чатового сценария взаимодействия с LLM, здесь результатом является не одиночный ответ на вопрос, а полноформатный персонализированный конспект, создаваемый в воспроизводимом и управляемом конвейере.

2.6 Выводы по главе

1 Во второй главе предложена архитектура сервиса персонализированных учебных конспектов, интегрируемого в платформу ЕГЭПрорыв [33] и использующего пользовательскую аналитику, RAG и LLM в рамках единого прикладного контура.

2 Обосновано, что применение RAG-подхода и векторной базы данных Qdrant позволяет связать генерацию конспекта с проверенными учебными материалами, повысить релевантность контекста и снизить риск предметных неточностей.

3 Показано, что персонализация достигается не только за счет генеративной модели, но прежде всего за счет анализа цифрового следа обучающегося, выделения устойчивых ошибок и последующей адаптации структуры объяснения к профилю ученика.

4 Предложенный алгоритм формирования конспекта обладает элементами научно-прикладной новизны, поскольку объединяет в одном воспроизводимом пайплайне learner modeling, retrieval по векторной базе знаний и управляемую генерацию структурированного образовательного текста.

5 Практическая ценность разработанного решения состоит в возможности промышленной интеграции сервиса в высоконагруженную образовательную платформу, где наряду с качеством генерации учитываются масштабируемость, наблюдаемость, асинхронная обработка и устойчивость к росту пользовательской нагрузки.

3 Глава 3. Реализация подсистемы извлечения и генерации персонализированных конспектов

В предыдущей главе были описаны архитектура сервиса и обоснование выбора основных методов. Данная глава раскрывает техническую реализацию ключевой подсистемы — RAG-пайплайна, отвечающего за извлечение релевантного учебного контекста и генерацию персонализированного конспекта. Последовательно рассматриваются: структура предметной базы знаний, модели векторного представления и механизм индексирования, алгоритм гибридного поиска с многоуровневым ранжированием, а также конвейер генерации и верификации итогового текста.

3.1 Структура предметной базы знаний

Предметная база знаний является центральным источником доверенного контента для RAG-пайплайна. В отличие от классического подхода к RAG, при котором корпус формируется путем автоматического разбиения (chunking) длинных документов на фрагменты фиксированного размера, в данной работе используется экспертно структурированная база знаний, состоящая из *атомов* — минимальных тематически целостных единиц учебного контента. Концепция атомов развивает идею *микромодулей*, предложенную в [34] и формализованную в [31]: каждый микромодуль представляет собой элементарную образовательную единицу, связанную с набором компетенций, а совокупность микромодулей образует ориентированный граф зависимостей. В нашей работе атомы сохраняют аналогичную графовую структуру пререквизитов, но дополнительно размечаются тегами ошибок и межзадачными навыками, что делает их пригодными не только для адаптации траектории обучения, но и для адресного извлечения контекста при генерации конспекта.

Каждый атом представляет собой запись со следующими полями:

1 **id** — уникальный идентификатор, кодирующий номер задания и порядковый номер (например, `t06_mm01`, `t10_mm51`, `t12_mm03`);

2 **title** — краткое название темы микромодуля;

3 **text** — учебный текст с формулами, определениями, примерами решений и указаниями на типичные ошибки;

4 **task_number** — номер задания ЕГЭ (6, 10 или 12);

5 **subtypes** — список подтипов задач, к которым относится атом (например, `exponential_eq`, `motion`, `polynomial_extrema`);

6 **error_tags** — перечень тегов типичных ошибок, связанных с данным атомом (например, `deriv_forgot_chain_rule`, `river_swap_plus_minus`);

7 **prerequisites** — список идентификаторов атомов-пререквизитов, образующих направленный ациклический граф зависимостей;

8 **shared_skills** — метки межзадачных навыков (например, `equation_setup`, `sign_analysis`), позволяющих связывать атомы из разных заданий ЕГЭ по общей компетенции.

Атомы хранятся в формате YAML в отдельных файлах для каждого задания: `atoms_t6.yaml`, `atoms_t10.yaml`, `atoms_t12.yaml`. Дополнительно предусмотрен файл `misconceptions.yaml`, содержащий экспертно описанные типичные заблуждения учеников. Каждая запись о заблуждении привязана к конкретному тегу ошибки и включает текстовое описание некорректного рассуждения, его причины и мнемоническое правило для запоминания верного подхода. Заблуждения загружаются в общий массив атомов и индексируются наравне с основным учебным контентом, что позволяет системе при обнаружении у ученика конкретной ошибки извлекать не только теоретический материал, но и адресное разъяснение заблуждения.

Совокупность атомов формирует направленный ациклический граф (DAG) пререквизитов. Связь «атом A является пререквизитом атома B» означает, что для понимания материала B необходимо предварительное усвоение материала A. Такая структура используется в процессе retrieval для двух целей. Во-первых, если для ученика релевантен атом B с высоким ранжирующим баллом, система автоматически добавляет пререквизитные атомы A в набор извлеченных фрагментов, обеспечивая логическую полноту объяснения. Во-вторых, при формировании итогового конспекта атомы упорядочиваются топологической сортировкой по графу пререквизитов, так что пререквизитные понятия предшествуют зависимым.

Помимо прямых пререквизитных связей, механизм *shared skills* обеспечивает межзадачный перенос (cross-task transfer). Ряд навыков — составление уравнения, проверка ограничений, анализ знака — используются одновременно в заданиях №6, №10 и №12, хотя предметный контекст различается. Четыре группы shared skills (`equation_setup`, `constraint_checking`, `sign_analysis`, `answer_plausibility`) объединяют атомы из разных заданий, позволяя профильному бусту учитывать успехи и ошибки ученика по смежным компетенциям.

Выбор атомарной структуры базы знаний вместо автоматического chunking обусловлен спецификой образовательного домена. Во-первых, каждый атом содержит тематически замкнутое объяснение, что устраняет проблему «разрезания» связного определения или примера на границе чанка. Во-вторых, экспертная разметка тегами ошибок и подтипами задач создает структуру метаданных, недоступную при автоматическом разбиении, и позволяет строить более точные фильтры на этапе поиска. В-третьих, граф пререквизитов обеспечивает логическую последовательность контекста, которая невозможна в плоском корпусе чанков.

3.2 Векторное представление и индексирование учебных материалов

Для обеспечения семантического поиска по базе знаний каждый атом представляется в виде двух типов векторных эмбеддингов: плотного (dense) и разреженного (sparse). Совместное использование двух представлений лежит в основе гибридного поиска, описанного в следующем разделе.

Плотные эмбеддинги. Для получения плотных векторных представлений используется модель `mxbai-embed-large-v1`, разработанная mixedbread.ai [19], с размерностью вектора 1024. Загрузка и инференс модели осуществляются через библиотеку fastembed [22], которая обеспечивает эффективную пакетную обработку без внешних API-зависимостей. После получения эмбеддингов выполняется L2-нормализация:

$$\hat{\mathbf{v}}_i = \frac{\mathbf{v}_i}{\max\bigl(\|\mathbf{v}_i\|_2,\ 10^{-12}\bigr)}, \tag{1}$$

где $\mathbf{v}_i$ — исходный вектор i-го документа. Нормализация гарантирует, что косинусное сходство между запросом и документом вычисляется как скалярное произведение нормализованных векторов.

Разреженные эмбеддинги. Для лексического компонента поиска используется модель `Qdrant/bm25` из библиотеки fastembed, реализующая BM25-подобное взвешивание термов [25]. Разреженное представление содержит индексы ненулевых термов и их весовые коэффициенты, что позволяет эффективно выполнять точное терм-совпадение по лексическому содержимому документов. Такое представление дополняет семантический поиск, обеспечивая высокую точность при совпадении ключевых терминов (например, «дискриминант», «логарифм»), которые могут быть размыты при исключительно семантическом сопоставлении.

Хранение в Qdrant. В качестве векторной базы данных используется Qdrant [23].

Коллекция создается с двумя именованными векторными пространствами:

1 `dense` — плотные векторы размерности 1024 с метрикой косинусного расстояния;

2 `sparse` — разреженные BM25-векторы переменной размерности.

Для каждой точки в коллекции сохраняется payload — набор метаданных, включающий идентификатор атома (`doc_id`), полный текст документа, номер задания (`task_number`), список подтипов (`subtypes`), теги ошибок (`error_tags`), заголовок атома и хеш содержимого (`content_hash`). Для полей `task_number` и `subtypes` создаются payload-индексы, что позволяет выполнять серверную фильтрацию при поиске без перебора всех точек коллекции.

Механизм инкрементального обновления. При инициализации RAG-движка вычисляется SHA-256 хеш от текстового содержимого всех атомов. Если в Qdrant уже существует коллекция с совпадающим числом точек, размерностью плотного вектора и хешем контента, повторное эмбеддирование и загрузка пропускаются. В противном случае коллекция пересоздается и данные загружаются пакетами. Такой подход сокращает время холодного старта сервиса при неизменной базе знаний и гарантирует консистентность индекса при обновлении контента.

Формирование текста документа. Перед эмбеддированием для каждого атома формируется текст документа в формате «заголовок + текст»: конкатенация полей `title` и `text` через символ новой строки. Заголовок включается в документ, поскольку содержит ключевые понятия (например, «Показательное уравнение с одинаковым основанием»), важные для семантического сопоставления с запросом.

3.3 Гибридный поиск и многоуровневое ранжирование

Процесс извлечения релевантного контекста реализован как многоэтапный пайплайн (Рисунок 3.1), включающий формирование запросов из профиля ученика, гибридный поиск в Qdrant, межзапросное слияние, профильный буст, расширение по графу пререквизитов, диверсификацию через MMR и топологическую сортировку.

Формирование поисковых запросов из профиля. В отличие от типичного RAG, где запрос формулируется пользователем, в данной системе множество поисковых запросов генерируется автоматически на основе профиля ученика. Стратегия зависит от наличия данных. Если в профиле присутствуют недавние неверные попытки, каждая из них преобразуется в текстовый запрос вида «ЕГЭ задание N: [условие задачи], ученик ответил X, правильно Y». Дополнительно добавляются запросы из пулов микромодулей (*micromodule*

queries), формируемые из текстов задач, в которых ученик допускал ошибки. При отсутствии данных об ошибках используется *cold start*-стратегия: детерминированная (по хешу идентификатора ученика) выборка из предзаданных пулов запросов для каждого задания. Совокупность запросов дедуплицируется и передается в retrieval.

Гибридный поиск в Qdrant. Для каждого текстового запроса вычисляются плотный и разреженный эмбеддинги, после чего выполняется гибридный поиск через механизм Qdrant Prefetch + RRF [23]. Запрос к Qdrant содержит два блока предварительной выборки (prefetch): один по плотному вектору с метрикой косинусного сходства, другой — по разреженному вектору. Каждый prefetch возвращает до $L \cdot p$ кандидатов, где L — целевое число результатов, p — множитель prefetch. Результаты двух потоков объединяются на сервере Qdrant с помощью взвешенного Reciprocal Rank Fusion (RRF) [8]:

$$\mathrm{RRF}(d) = \frac{w_{\mathrm{dense}}}{k + r_{\mathrm{dense}}(d)} + \frac{w_{\mathrm{sparse}}}{k + r_{\mathrm{sparse}}(d)}, \tag{2}$$

где $r_{\mathrm{dense}}(d)$ и $r_{\mathrm{sparse}}(d)$ — ранги документа d в плотном и разреженном списках соответственно, k — константа сглаживания (по умолчанию $k = 60$), w_{dense} и w_{sparse} — весовые коэффициенты каналов (по умолчанию $w_{\mathrm{dense}} = 0{,}40$, $w_{\mathrm{sparse}} = 0{,}25$). При наличии фильтров по номеру задания или подтипу задачи серверная фильтрация применяется до ранжирования, что существенно сокращает множество кандидатов.

Межзапросное RRF-слияние (RAG-Fusion). Поскольку для одного ученика генерируется несколько поисковых запросов, результаты гибридного поиска по всем запросам объединяются вторым уровнем RRF:

$$S_{\mathrm{fused}}(d) = \sum_{j=1}^{Q} \frac{1}{k + r_j(d)}, \tag{3}$$

где Q — число запросов, $r_j(d)$ — ранг документа d в результатах j-го запроса. Документы, встречающиеся в результатах нескольких запросов, получают более высокий итоговый балл, что естественным образом отражает устойчивость релевантности фрагмента для данного ученика.

Профильный буст. После межзапросного слияния итоговый балл каждого документа корректируется на основе профиля ученика:

$$S(d) = S_{\mathrm{fused}}(d) + \gamma \cdot \Big(\mathrm{err}(d) + \beta \cdot \mathrm{skill}(d)\Big) - \mathbb{1}\big[m(d) > \theta_m\big] \cdot \delta_m, \tag{4}$$

где γ — вес профильного компонента ($\gamma = 0{,}30$); $\mathrm{err}(d)$ — суммарный балл ошибок по тегам, связанным с атомом d; $\mathrm{skill}(d)$ — суммарный балл по shared skills атома; β — множитель навыковой компоненты ($\beta = 0{,}3$); $m(d)$ — уровень освоения атома; θ_m — порог mastery ($\theta_m = 0{,}8$); δ_m — штраф за освоенный атом ($\delta_m = 0{,}15$).

Балл ошибки $\mathrm{err}(d)$ вычисляется по журналу событий профиля с применением степенного затухания (power-law decay):

$$\mathrm{error_score}(t) = \sum_{e \in \mathcal{E}_t} w_e \cdot (1 + \Delta_e)^{-\alpha}, \tag{5}$$

где $\mathcal{E}_t$ — множество событий ошибки с тегом t, w_e — вес события, Δ_e — разность между текущим номером попытки и номером попытки события, $\alpha = 0{,}5$ — параметр затухания. Степенной закон обеспечивает постепенное забывание старых ошибок, при этом недавние ошибки вносят доминирующий вклад в балл. Данный механизм перекликается с экспоненциальной кривой забывания, используемой в адаптивном модуле платформы для оценки вероятности владения компетенцией [34]: там $p_k(t) = p_k(t_0) \cdot e^{-\lambda(t-t_0)}$ моделирует спад владения компетенцией при отсутствии практики, а в [31] этот подход дополнен кластеризацией родственных задач и пересчётом вероятностей после каждой попытки. В отличие от этих задач, наш decay применяется не к вероятности владения, а к *весу* ошибки при ранжировании контента: цель состоит не в том, чтобы оценить уровень знаний, а в том, чтобы обеспечить приоритет актуальных проблем в контексте конспекта. Механизм mastery gating подавляет ранжирование атомов, которые ученик, по оценке системы, уже достаточно освоил, направляя контекст конспекта на актуальные пробелы.

Расширение по графу пререквизитов. Для обеспечения логической полноты конспекта система выполняет расширение набора кандидатов по графу атомов. Для каждого из топ-k атомов в текущем рейтинге извлекаются пререквизиты до глубины 2 (обход BFS) и непосредственные зависимые (dependents). Пререквизит получает бонус к рангу, пропорциональный баллу родительского атома, с учетом «слабости» (atom weakness) ученика по данному пререквизиту:

$$S(p) \leftarrow \max\bigl(S(p),\ S(a) + b_{\mathrm{prereq}} \cdot (1 + w_p) + b_{\mathrm{tag}} \cdot |\mathrm{tags}(p)|\bigr), \tag{6}$$

где a — родительский атом, p — пререквизит, b_{prereq} и b_{tag} — конфигурируемые бонусы, w_p — слабость ученика по атому p. Для пререквизитов из другого задания ЕГЭ (cross-task) применяется понижающий множитель. Зависимые атомы (dependents) получают фиксированный

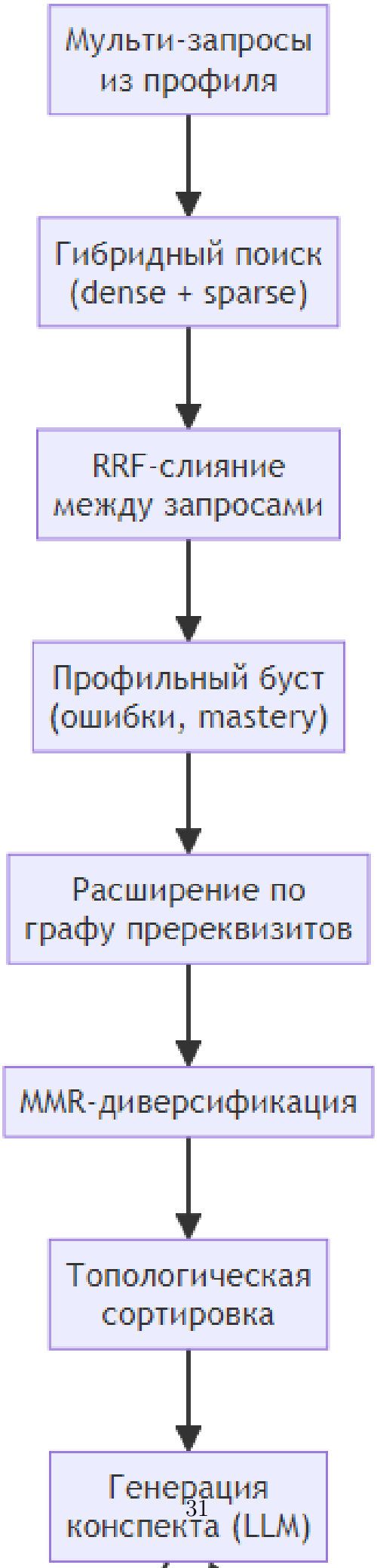
бонус, если удовлетворяют фильтрам по заданию и подтипу.

MMR-диверсификация. После объединения базовых и расширенных кандидатов финальный отбор выполняется алгоритмом Maximal Marginal Relevance [6]:

$$\mathrm{MMR}(d_i) = \lambda \cdot \hat{S}(d_i) - (1 - \lambda) \cdot \max_{d_j \in \mathcal{S}} \mathrm{sim}(d_i, d_j), \tag{7}$$

где $\hat{S}(d_i)$ — нормализованная в $[0, 1]$ релевантность, $\mathcal{S}$ — уже отобранные атомы, $\mathrm{sim}(d_i, d_j)$ — сходство Жаккара по множествам подтипов задач, $\lambda = 0{,}6$. MMR-отбор обеспечивает тематическое разнообразие конспекта: если в топе ранжирования оказалось несколько атомов одного и того же подтипа, алгоритм отдает предпочтение атомам из менее представленных подтипов, сохраняя баланс между релевантностью и полнотой охвата.

Топологическая сортировка. Итоговый набор из k отобранных атомов (по умолчанию $k = 12$) упорядочивается топологической сортировкой по графу пререквизитов. Алгоритм выполняет рекурсивный обход: для каждого атома сначала посещаются его пререквизиты, уже входящие в отобранный набор, затем сам атом. В результате в контексте промпта базовые понятия предшествуют зависимым, что воспроизводит дидактически обоснованную последовательность изложения.


Мульти-запросы из профиля
Гибридный поиск (dense + sparse)
RRF-слияние между запросами
Профильный буст (ошибки, mastery)
Расширение по графу пререквизитов
MMR-диверсификация
Топологическая сортировка
Генерация конспекта (LLM)

3.4 Генерация и верификация конспекта

На основе профиля ученика и извлеченного контекста RAG-пайплайн переходит к этапу генерации текста конспекта. Этот этап включает сборку структурированного промпта, вызов языковой модели, верификацию математического содержания и постобработку результата.

Сборка промпта. Промпт конспекта состоит из системного сообщения и пользовательского сообщения. Оба формируются на основе шаблонов, хранящихся в YAML-файле `conspect_prompts.yaml`. Пользовательское сообщение включает:

1 указание на задание и подтип;

2 уровень детализации, определяемый автоматически по точности ответов ученика (подробный при точности ниже 30%, средний — от 30% до 60%, краткий — выше 60%);

3 блок ошибок ученика с расшифровкой до 5 наиболее значимых тегов ошибок в человекочитаемом формате, конкретными примерами неверных ответов и условиями задач;

4 извлеченный контекст, разделенный на справочный материал и блок типичных заблуждений;

5 темы-фронтиры — атомы, для которых ученик демонстрирует слабость, но пререквизиты уже освоены, что указывает на зону ближайшего развития;

6 инструкцию по разнообразию примеров, предотвращающую генерацию шаблонных задач.

При включенном режиме структурированного вывода (structured output) к системному промпту добавляется JSON-инструкция, предписывающая модели возвращать конспект в виде объекта с именованными секциями. Это позволяет применять верификацию к каждой секции отдельно.

Генерация. Для генерации конспекта используется языковая модель, доступная через один из поддерживаемых бэкендов: Groq, OpenRouter или Ollama. Выбор бэкенда определяется конфигурацией, что позволяет переключаться между облачными и локальными моделями без изменения логики пайплайна. Генерация выполняется с температурой $T = 0{,}25$ и ограничением длины 8192 токена.

Верификация и ремонт. После генерации черновика конспект проходит многоступенчатую верификацию:

1 **Структурированная верификация (optional).** Если генерация выполнена в JSON-режиме, каждая математическая секция (формулы, примеры, чеклист) проверяется отдельным вызовом LLM с системным промптом, настроенным на поиск арифметических и алгебраических ошибок.

2 **Полнотекстовая верификация.** Второй вызов LLM с низкой температурой ($T = 0{,}1$) проверяет весь конспект на полноту обязательных смысловых блоков, предметную корректность и педагогическую согласованность.

3 **Математическая верификация.** Отдельный проход проверяет вычисления, знаки и формулы на фактическую корректность.

4 **Валидация формата.** Регулярные выражения проверяют конспект на наличие нарушений формата ЕГЭ: ответы в виде десятичных дробей вместо обыкновенных, некорректные обозначения, пропущенные единицы измерения. Обнаруженные нарушения передаются LLM для автоматического ремонта с сохранением структуры и заголовков конспекта.

Постобработка. Финальная постобработка включает удаление из текста конспекта внутренних идентификаторов атомов, технических терминов пайплайна (если они «просочились» из контекста в генерацию), а также блоков «мышления» модели. При обнаружении в финальном ответе JSON-формата выполняется принудительная конвертация в Markdown, поскольку пользователь должен получать конспект исключительно в читаемом текстовом формате.

3.5 Выводы по главе

1 Предметная база знаний реализована как экспертно структурированный набор атомов с графом пререквизитов, тегами ошибок и межзадачными навыками, что обеспечивает тематическую целостность единиц поиска и возможность адресного извлечения контента по профилю ученика.

2 Двухканальное векторное представление (плотные эмбеддинги `mxbai-embed-large-v1` и разреженные BM25-эмбеддинги) в сочетании с хранением в Qdrant обеспечивает гибридный поиск, объединяющий семантическую и лексическую релевантность.

3 Многоуровневый пайплайн ранжирования — гибридный RRF в Qdrant, межзапросное RRF-слияние, профильный буст с power-law decay, расширение по графу пререквизи-

тов и MMR-диверсификация — позволяет формировать контекст, адаптированный к текущему состоянию подготовки конкретного ученика, а не к усредненному запросу.

4 Конвейер генерации с многоступенчатой верификацией (структурная, полнотекстовая, математическая и форматная проверки) снижает вероятность предметных неточностей и обеспечивает соответствие итогового конспекта формату ЕГЭ.

5 Все параметры RAG-пайплайна (веса каналов, константа RRF, бонусы графа, порог mastery, параметр затухания) вынесены в конфигурацию и могут настраиваться через переменные окружения, что обеспечивает воспроизводимость экспериментов и возможность тонкой настройки без изменения кода.

6 Экспериментальные прогоны генерации конспектов выполнялись на основе анонимизированной выгрузки решений учеников с портала ЕГЭПрорыв (выборка из 36 обучающихся; персональные данные удалены в соответствии с требованиями 152-ФЗ); пример результата приведён в приложении.

4 Примеры

4.1 Ссылки на статьи

Ссылки на статьи оформляются с помощью пакета `biblatex`, например [13]. В описании статье в bib файле нужно обязательно указывать место публикации работы (журнал или конференцию) и год. Обратите внимание, что для описания статей из разных источников в списке литературы используются разные команды в bib файле: статья из журнала [32], статья с конференции [36], книга [35], глава книги [18]. Если статься еще не опубликована нигде, а только выложена на arXiv, то на нее тоже можно сослаться [12], но предпочтительно ссылаться на опубликованную версию, если она уже существует. Если вы хотите сослаться на сайт, то можно либо так же внести его в список литературы [3] (рекомендуется, если таких ссылок у вас много из-за особенностей темы вашего проекта), либо использовать ссылку внизу страницы [1]. При работе с онлайн ресурсами не забывайте указывать дату обращения к этому ресурсу, так как в отличие от опубликованных статей, эти ресурсы могут измениться в любой момент.

4.2 Рисунки

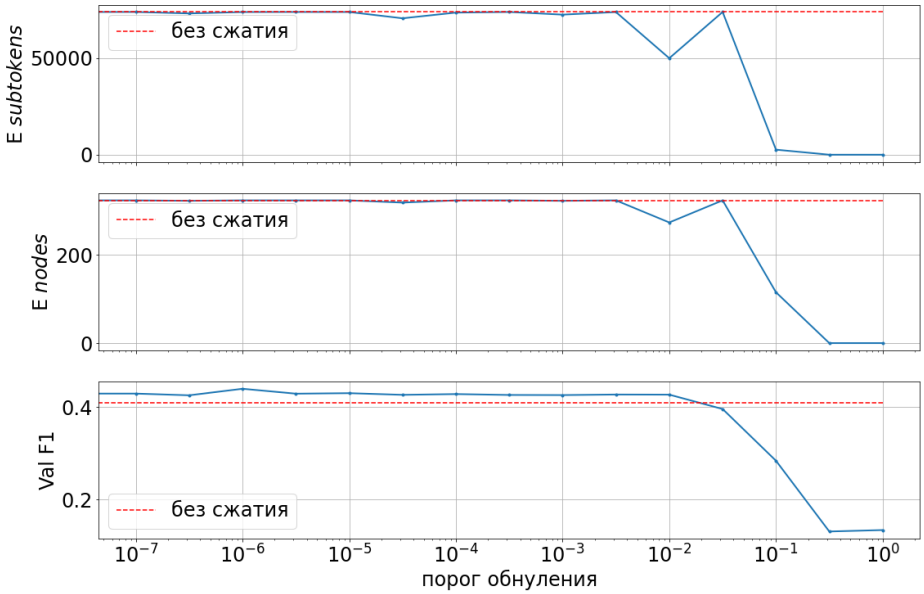


Рис. 4.1: Пример графика. Тут должна быть подпись, поясняющая что происходит на рисунке (краткая, но достаточная для понимания основной идеи графика).

[1] Книги доступны по ссылке: `http://www-cs-faculty.stanford.edu/~uno/abcde.html`, дата обр. 16.05.2013

Все рисунки в тексте должны иметь подписи и вы на них должны ссылаться в тексте. Например, на Рисунке 4.1 изображен пример графика. Не забывайте подписывать все оси на графиках, добавлять легенду и пояснять все обозначения, а также используйте адекватного размера шрифты и толщину линий на графиках (все должно быть видно и понятно без многократного увеличения). На рисунке из примера явно не хватает обозначения синей линии в легенде.

4.3 Таблицы

Все таблицы в тексте тоже должны иметь подписи и вы на них должны ссылаться в тексте. Например, в Таблице 4.1 показаны результаты примерного эксперимента.

Таблица 4.1: Пример таблички. Тут должна быть подпись, поясняющая что происходит в таблице (краткая, но по делу).

	Val			Test				
	Prec	Rec	F1	Prec	Rec	F1	nodes	subtokens
запуск 1	0.4894	0.3775	0.4263	0.4824	0.3683	0.4177	10029	179
запуск 2	0.4887	0.3739	0.4237	0.4891	0.3724	0.4228	10039	177
запуск 3	0.4820	0.3751	0.4219	0.4838	0.3677	0.4178	10037	180
среднее	**0.4867**	**0.3755**	**0.4239**	**0.4851**	**0.3695**	**0.4195**		
дисперсия	0.0041	0.0019	0.0022	0.0036	0.0025	0.0029		

4.4 Формулы

Формулы стоит центрировать, а также нумеровать, если вы ссылаете на них в тексте. Также не забывайте пояснять все обозначения в формулах. Например, запишем следующую задачу оптимизации:

$$\theta* = \min_{\theta} F(\theta), \tag{8}$$

где F – квадратичная функция от параметра θ. При необходимости, далее в тексте можно сослаться на формулу (8). При этом, в зависимости от конкретных формул, можно использовать разные слова: формула, уравнение, задача оптимизации и т.п.

Список литературы

1. Ghodai Abdelrahman, Qing Wang и Bernardo Nunes. “Knowledge Tracing: A Survey”. В: *ACM Computing Surveys* 55.11 (2023), с. 1—37. DOI: 10.1145/3569576.

2. Riordan Alfredo, Vanessa Echeverría, Yueqiao Jin, Lixiang Yan, Zachari Swiecki, Dragan Gašević и Roberto Martínez-Maldonado. “Human-centred Learning Analytics and AI in Education: A Systematic Literature Review”. В: *Computers and Education: Artificial Intelligence* 6 (2024), с. 100215. DOI: 10.1016/j.caeai.2024.100215.

3. Anthropic. *Model Context Protocol: An open standard for connecting AI assistants to data*. Англ. Дата обращения: 30.03.2026. 2025. URL: https://www.anthropic.com/research/model-context-protocol.

4. Ryan S. J. d. Baker. “Data Mining for Education”. В: *International Encyclopedia of Education*. Под ред. Barry McGaw, Penelope Peterson и Eva Baker. 3rd. Elsevier, 2010.

5. Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel M. Ziegler, Jeffrey Wu, Clemens Winter, Christopher Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever и Dario Amodei. “Language Models are Few-Shot Learners”. В: *Advances in Neural Information Processing Systems (NeurIPS)*. Т. 33. 2020, с. 1877—1901.

6. Jaime Carbonell и Jade Goldstein. “The Use of MMR, Diversity-Based Reranking for Reordering Documents and Producing Summaries”. В: *Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1998, с. 335—336. DOI: 10.1145/290941.291025.

7. Albert T. Corbett и John R. Anderson. “Knowledge Tracing: Modeling the Acquisition of Procedural Knowledge”. В: *User Modeling and User-Adapted Interaction* 4.4 (1994), с. 253—278. DOI: 10.1007/BF01099821.

8. Gordon V. Cormack, Charles L. A. Clarke и Stefan Buettcher. “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods”. В: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2009, с. 758—759. DOI: 10.1145/1571941.1572114.

9. Jacob Devlin, Ming-Wei Chang, Kenton Lee и Kristina Toutanova. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. В: *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*. 2019, с. 4171—4186.

10. Yixuan Ding, Jingheng Feng, Haochen Guo, Minjie Xu, Xinai Lu и Jiaxin Zhang. “Retrieval-Augmented Generation for Educational Application: A Systematic Survey”. В: *Computers and Education: Artificial Intelligence* 8 (2025), с. 100378. DOI: 10.1016/j.caeai.2025.100378.

11. Chevy van Dorresteijn, Dina Fajardo-Tovar, Natalie Pareja Roblin, Frank Cornelissen, Monique Meij, Joke Voogt и Monique Volman. “What Factors Contribute to Effective Online Higher Education? A Meta-Review”. В: *Technology, Knowledge and Learning* 30 (2025), с. 171—202. DOI: 10.1007/s10758-024-09750-5.

12. Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang и Haofen Wang. “Retrieval-Augmented Generation for Large Language Models: A Survey”. В: *arXiv preprint arXiv:2312.10997* (2024).

13. Enkelejda Kasneci, Kathrin Sessler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Jürgen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, Jochen Weller, Jochen Kuhn и Gjergji Kasneci. “ChatGPT for Good? On Opportunities and Challenges of Large Language Models for Education”. В: *Learning and Individual Differences* 103 (2023), с. 102274. DOI: 10.1016/j.lindif.2023.102274.

14. Khan Academy. *Khanmigo: AI-powered learning guide*. Англ. Дата обращения: 30.03.2026. 2025. URL: https://www.khanacademy.org/khanmigo.

15. Ean Teng Khor и Mutthulakshmi K. “A Systematic Review of the Role of Learning Analytics in Supporting Personalized Learning”. В: *Education Sciences* 14.1 (2024), с. 51. DOI: 10.3390/educsci14010051.

16. LangChain. *LangGraph*. Англ. Дата обращения: 30.03.2026. 2024. URL: https://blog.langchain.com/langgraph/.

17. Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel и

Douwe Kiela. “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks”. В: *Advances in Neural Information Processing Systems (NeurIPS)*. Т. 33. 2020, с. 9459—9474.

18. MAXIMUM. *Курсы подготовки к ЕГЭ: онлайн-обучение для 11 класса*. Дата обращения: 30.03.2026. 2026. URL: https://maximumtest.ru/ege.

19. mixedbread.ai. *mxbai-embed-large-v1: Open Source Embedding Model*. Дата обращения: 01.04.2026 2024. URL: https://huggingface.co/mixedbread-ai/mxbai-embed-large-v1.

20. Maciej Pankiewicz и Ryan S. Baker. “Generating In-Context, Personalized Feedback for Intelligent Tutors with Large Language Models”. В: *International Journal of Artificial Intelligence in Education* (2025). DOI: 10.1007/s40593-025-00505-6.

21. Chris Piech, Jonathan Bassen, Jonathan Huang, Surya Ganguli, Mehran Sahami, Leonidas Guibas и Jascha Sohl-Dickstein. “Deep Knowledge Tracing”. В: *Advances in Neural Information Processing Systems (NeurIPS)*. Т. 28. 2015.

22. Qdrant. *fastembed: Fast, Accurate, Lightweight Python Library for Embedding Generation*. Дата обращения: 01.04.2026. 2026. URL: https://github.com/qdrant/fastembed.

23. Qdrant. *Qdrant Documentation*. Англ. Дата обращения: 31.03.2026. 2026. URL: https://qdrant.tech/documentation/overview/.

24. Quizlet. *Introducing Q-Chat: The first AI tutor built with OpenAI’s API*. Англ. Дата обращения: 30.03.2026. 2025. URL: https://quizlet.com/blog/introducing-q-chat.

25. Stephen Robertson и Hugo Zaragoza. “The Probabilistic Relevance Framework: BM25 and Beyond”. В: *Foundations and Trends in Information Retrieval* 3.4 (2009), с. 333—389. DOI: 10.1561/1500000019.

26. Cristóbal Romero и Sebastián Ventura. “Educational Data Mining and Learning Analytics: An Updated Survey”. В: *WIREs Data Mining and Knowledge Discovery* 10.3 (2020), e1355. DOI: 10.1002/widm.1355.

27. Yuhong Shi, Kun Yu, YiFei Dong и Fang Chen. “Large Language Models in Education: A Systematic Review of Empirical Applications, Benefits, and Challenges”. В: *Computers and Education: Artificial Intelligence* 10 (2025), с. 100529. DOI: 10.1016/j.caeai.2025.100529.

28. Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, Aurelien Rodriguez, Armand Joulin, Edouard Grave и Guillaume Lample. “LLaMA: Open and Efficient Foundation Language Models”. В: *arXiv preprint arXiv:2302.13971* (2023).

29. Ang Wang, Lin Chen и Yuhan Wang. “TutorLLM: Customizing Learning Recommendations with Knowledge Tracing and Retrieval-Augmented Generation”. В: *arXiv preprint arXiv:2502.1570* (2025).

30. Lixiang Yan, Lele Sha, Linxuan Zhao, Yuheng Li, Roberto Martinez-Maldonado, Guanliang Chen, Xinyu Li, Yueqiao Jin и Dragan Gašević. “Practical and Ethical Challenges of Large Language Models in Education: A Systematic Scoping Review”. В: *British Journal of Educational Technology* 55.1 (2024), с. 90—112. DOI: 10.1111/bjet.13370.

31. Роман Игоревич Агарев. *Оптимизация образовательных траекторий с применением машинного обучения в онлайн-курсах*. Отчёт об исследовательском проекте. Науч. рук. О. В. Тарасов. НИУ ВШЭ, Факультет компьютерных наук, 2025.

32. ЕГЭLand. *Онлайн-школа подготовки к ЕГЭ и ОГЭ ЕГЭLand*. Дата обращения: 30.03.2026. 2026. URL: https://el-ed.ru/.

33. ЕГЭПрорыв. *ЕГЭПрорыв — онлайн-школа подготовки к ЕГЭ и ОГЭ*. Дата обращения: 30.03.2026. 2026. URL: https://exambreakout.ru/.

34. Олег Васильевич Тарасов и Никита Алексеевич Птицын. “Разработка системы адаптивного обучения учащихся старшей школы”. Направление 27.04.07 Наукоёмкие технологии и экономика инноваций. Магистерская диссертация. Московский физико-технический институт (национальный исследовательский университет), 2025.

35. Умскул. *Умскул — онлайн-школа подготовки к ЕГЭ и ОГЭ*. Дата обращения: 30.03.2026. 2026. URL: https://umschool.net/.

36. Яндекс. *Подготовка к ЕГЭ по математике (Яндекс.Образование)*. Дата обращения: 30.03.2026. 2026. URL: https://education.yandex.ru/ege/go/math.

A Пример сгенерированного конспекта

Ниже приведён фрагмент конспекта, сформированного RAG-сервисом для одного из анонимных профилей ученика (идентификатор `archive_u5`) на основе выборки решений с портала ЕГЭПрорыв. Исходные данные о попытках решения задач получены из деперсонализированной выгрузки (36 учеников; удалены ФИО, адреса электронной почты и иные идентификаторы). Полный текст конспекта в формате Markdown сохранён в репозитории разработки как часть архива экспериментальных прогонов.

Введение (фрагмент)

Давайте разберём два важных задания первой части ЕГЭ. Задание №10 — прикладная математика (подстановка в формулу), а №12 — исследование функции с помощью производной. В обоих случаях ответом должно быть целое число или конечная десятичная дробь.

Что важно запомнить (фрагмент)

1 **Производная логарифма:** $(\ln x)' = \frac{1}{x}$**.** Логарифм растёт всё медленнее по мере увеличения x. Если в №12 видишь функцию с ln, помни про ОДЗ: аргумент логарифма всегда > 0.

2 **Производная частного:** $\left(\frac{f}{g}\right)' = \frac{f'g - fg'}{g^2}$. В формуле производной частного в числителе всегда стоит минус перед слагаемым fg'.

3 **Правило «догона» (преследование с форой):** $t = \frac{S}{v_2 - v_1}$ — время догона при разнице скоростей.

Типичные ошибки (фрагмент)

1 **Игнорирование ограничений (ОДЗ) в текстовых задачах.** При решении квадратного уравнения в №10 могут получиться два корня; скорость, время и длина не могут быть отрицательными — выбирай физически осмысленный корень.

2 **Забытый минус в производной частного.** Путать знак с производной произведения нельзя: в числителе всегда $f'g - fg'$.

Разбор примера: задание №12 (фрагмент)

Условие. Найдите наименьшее значение функции $f(x) = x^2 - 8x + 19$ на отрезке $[1; 5]$.

Решение.

1 Производная: $f'(x) = 2x - 8$.

2 Критическая точка: $2x - 8 = 0 \Rightarrow x = 4$.

3 Проверка: $4 \in [1; 5]$.

4 Значения: $f(1) = 12$, $f(5) = 4$, $f(4) = 3$.

5 Наименьшее из $\{12; 4; 3\}$ равно 3.

Ответ: 3.

Чеклист перед ответом (фрагмент)

- В ответе нет букв, корней, π и обыкновенных дробей (если этого требует формат задания).
- В задаче №12 вычислены значения функции на концах отрезка и в критических точках внутри отрезка.
- Единицы измерения в текстовой задаче №10 приведены к одной системе.